LAPORAN PRAKTIKUM TUGAS 1: RESTFUL API MURNI DENGAN EXPRESS.JS
PENGEMBANGAN APLIKASI WEB II
KEBUN



Dosen Pengampu: Nur Rachmat, M.Kom.

Oleh:

Deisy Callisca Chandra **2428240054**

Topik : Kebun

Kelas SI5B

FAKULTAS ILMU KOMPUTER DAN REKAYASA
UNIVERSITAS MULTI DATA PALEMBANG
2026/2027

TUJUAN PRAKTIKUM

Praktikum ini bertujuan untuk:

1. Memahami konsep RESTful API dan penerapannya menggunakan Express.js.
2. Membuat RESTful API murni dengan Express.js untuk mengelola satu resource, yaitu tanaman.
3. Menerapkan operasi CRUD menggunakan method HTTP GET, POST, PUT, dan DELETE serta filter menggunakan query string.
4. Melakukan validasi request, memberikan HTTP status code yang sesuai, menguji API menggunakan Postman, serta melakukan deployment aplikasi ke Vercel.

DASAR TEORI

2.1 RESTful API

RESTful API merupakan layanan yang digunakan untuk mengelola suatu resource melalui URL dan method HTTP. Pada praktikum ini, resource yang digunakan adalah **tanaman** dengan endpoint utama /plants.

API memberikan response dalam format JSON sehingga data dapat digunakan oleh client tanpa perlu menghasilkan halaman HTML.

2.2 Resource

Resource merupakan objek atau data yang dikelola oleh API. Pada praktikum ini, resource yang digunakan adalah **tanaman**.

Resource tanaman memiliki beberapa atribut, yaitu:

- id
- namaTanaman
- jenis
- lokasiBedeng
- tanggalTanam
- sudahPanen

2.3 HTTP Method

RESTful API menggunakan beberapa HTTP method untuk melakukan operasi terhadap resource.

Method	**Fungsi**
GET	Mengambil data
POST	Menambahkan data
PUT	Mengubah data

Method	Fungsi
DELETE	Menghapus data

Pada API yang dibuat, method tersebut diterapkan pada endpoint /plants.

2.4 HTTP Status Code

HTTP status code digunakan untuk menunjukkan hasil dari request yang dikirimkan client.

Status code yang digunakan dalam praktikum ini antara lain:

- **200 OK** untuk request yang berhasil.
- **201 Created** untuk data yang berhasil ditambahkan.
- **400 Bad Request** untuk request dengan data yang tidak valid atau field wajib yang tidak terpenuhi.
- **404 Not Found** apabila resource atau endpoint tidak ditemukan.

2.5 JSON

JSON digunakan sebagai format pertukaran data antara client dan server. Pada praktikum ini seluruh response API dikembalikan dalam format JSON menggunakan res.json() atau res.status(...).json().

2.6 Express.js

Express.js merupakan framework Node.js yang digunakan untuk membangun aplikasi web dan API. Pada praktikum ini Express.js digunakan untuk membuat routing, middleware, menerima request JSON, memberikan response JSON, serta menjalankan RESTful API.

Sumber:
Dokumentasi materi Express.js dosen:
https://rachmat-nur.gitbook.io/express

ALAT DAN BAHAN

Alat dan bahan yang digunakan dalam praktikum ini adalah:

1. Node.js LTS
2. npm
3. Express.js
4. Nodemon
5. Visual Studio Code
6. Postman
7. Git
8. GitHub
9. Vercel
10. Web browser

LANGKAH PRAKTIKUM

4.1 Inisialisasi Project

Project dibuat dengan nama:

tugas1-restful-2428240054

Project menggunakan Node.js dan Express.js.

Struktur project yang digunakan adalah:

```
tugas1-restful-2428240054/
├── .gitattributes
├── .gitignore
├── app.js
├── package-lock.json
├── package.json
├── README.md
└── vercel.json
```

Folder node_modules/ digunakan oleh npm untuk menyimpan dependency dan tidak diikutsertakan dalam repository GitHub karena telah dimasukkan ke dalam .gitignore.

Instalasi Express

Perintah yang digunakan: npm install express

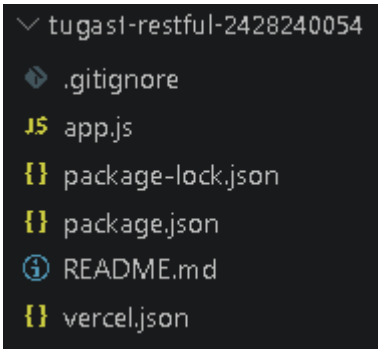
Instalasi Nodemon

Perintah yang digunakan: npm install --save-dev nodemon

Menjalankan aplikasi

Untuk menjalankan aplikasi dalam mode development digunakan: npm run dev

Screenshot 1. Struktur project



Screenshot 2. Proses menjalankan server



4.2 Konfigurasi package.json

File package.json digunakan untuk mengatur informasi project, dependency, dan script yang digunakan untuk menjalankan aplikasi.

Script yang digunakan adalah:

```
"scripts": {
  "start": "node app.js",
  "dev": "nodemon app.js"
}
```

Script dev digunakan agar aplikasi dapat dijalankan menggunakan Nodemon selama proses pengembangan.

Screenshot 3. Isi package.json



4.3 Data Awal

Data tanaman disimpan dalam array di dalam memory sesuai dengan ketentuan tugas.

Data awal yang digunakan terdiri dari minimal tiga tanaman.

Contoh data:

```
let plants = [
```

```
  {
    id: 1,
    namaTanaman: "Kangkung",
    jenis: "sayur",
    lokasiBedeng: "Bedeng 3",
    tanggalTanam: "2026-09-01",
    sudahPanen: false
  },
  {
    id: 2,
    namaTanaman: "Cabai Merah",
    jenis: "sayur",
    lokasiBedeng: "Bedeng 1",
    tanggalTanam: "2026-08-20",
    sudahPanen: true
  },
  {
    id: 3,
    namaTanaman: "Mangga Harum Manis",
    jenis: "buah",
    lokasiBedeng: "Bedeng 5",
    tanggalTanam: "2026-07-15",
    sudahPanen: false
  }
];
```

Data disimpan dalam array karena tugas tidak menggunakan database.

Screenshot 4. Data awal pada app.js

```
JS app.js > ...
  7   // Data awal tanaman
  8   let plants = [
  9       {
 10           id: 1,
 11           namaTanaman: "Kangkung",
 12           jenis: "sayur",
 13           lokasiBedeng: "Bedeng 3",
 14           tanggalTanam: "2026-09-01",
 15           sudahPanen: false
 16       },
 17       {
 18           id: 2,
 19           namaTanaman: "Cabai Merah",
 20           jenis: "sayur",
 21           lokasiBedeng: "Bedeng 1",
 22           tanggalTanam: "2026-08-20",
 23           sudahPanen: true
 24       },
 25       {
 26           id: 3,
 27           namaTanaman: "Mangga Harum Manis",
 28           jenis: "buah",
 29           lokasiBedeng: "Bedeng 5",
 30           tanggalTanam: "2026-07-15",
 31           sudahPanen: false
 32       }
 33   ];
```

4.4 GET /plants

Endpoint: GET /plants

digunakan untuk mengambil seluruh data tanaman.

Request dapat diuji menggunakan: http://localhost:3000/plants

Response yang diberikan berupa array JSON yang berisi seluruh data tanaman.

Screenshot 5. GET /plants



4.5 GET /plants/:id

Endpoint: GET /plants/:id

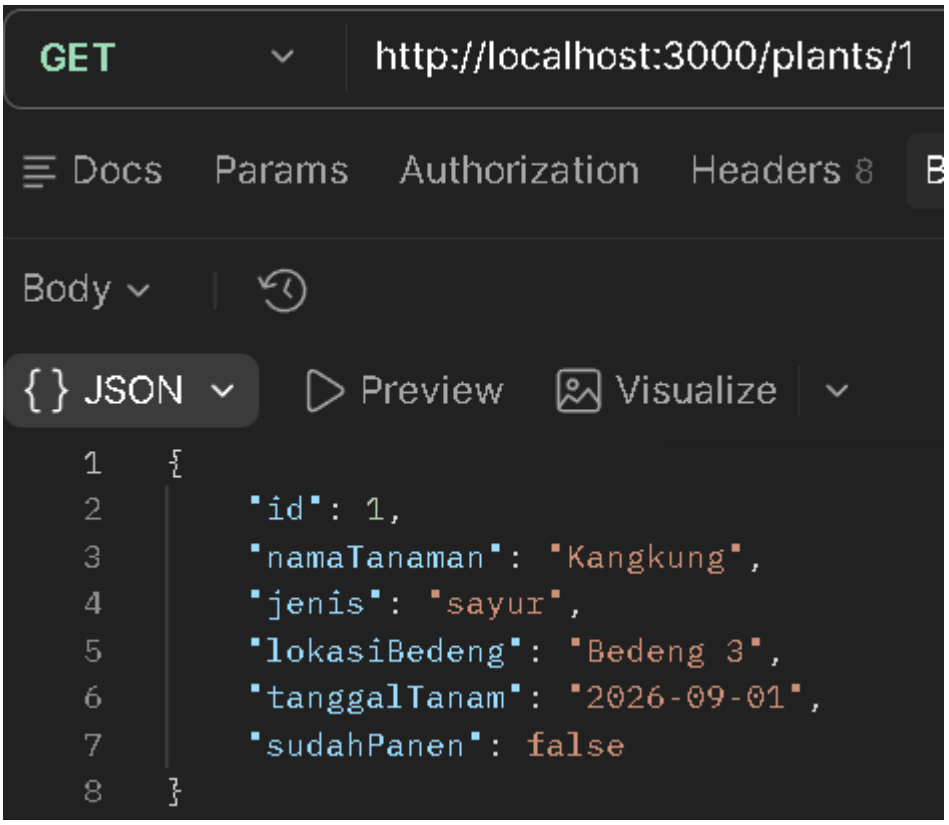
digunakan untuk mengambil satu data tanaman berdasarkan ID.

Contoh: GET /plants/1

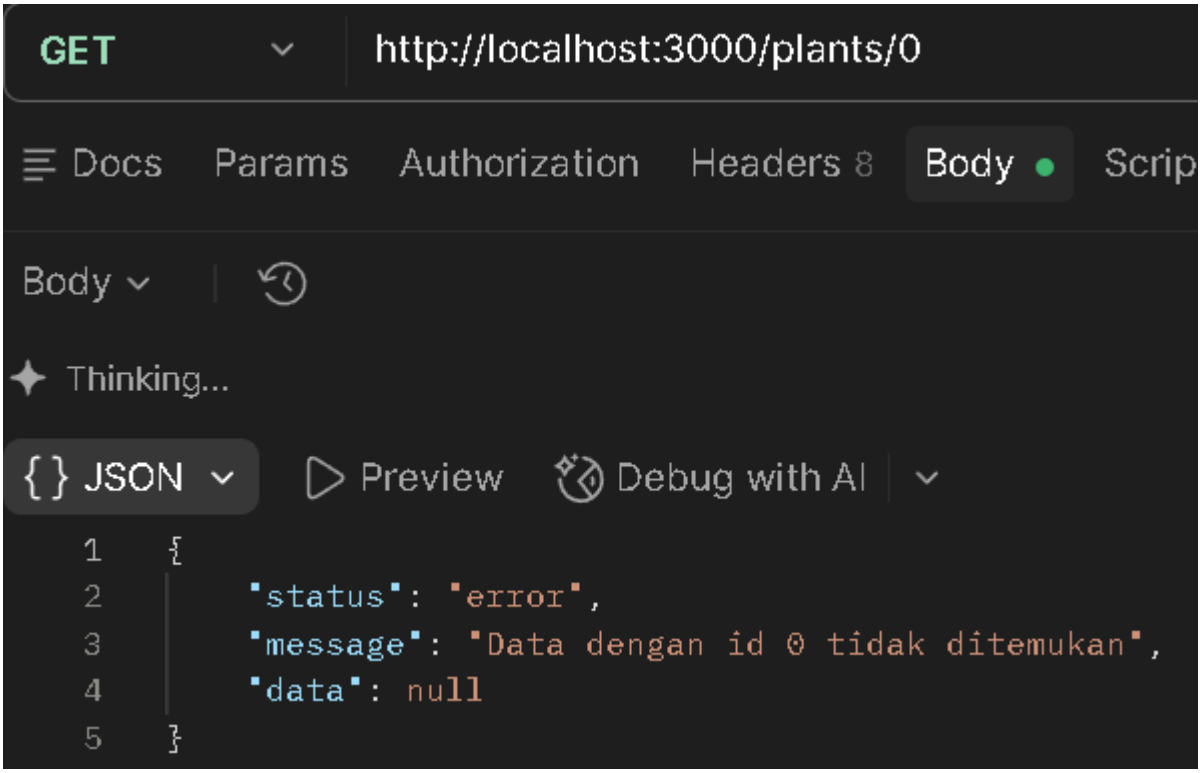
Apabila tanaman dengan ID tersebut ditemukan, API mengembalikan data tanaman.

Apabila ID tidak ditemukan, API memberikan response dengan status 404 Not Found.

Screenshot 6. GET /plants/



Screenshot 7. GET /plants/ dengan ID yang tidak ditemukan



4.6 Filter Tanaman

API menyediakan filter berdasarkan jenis tanaman menggunakan query string.

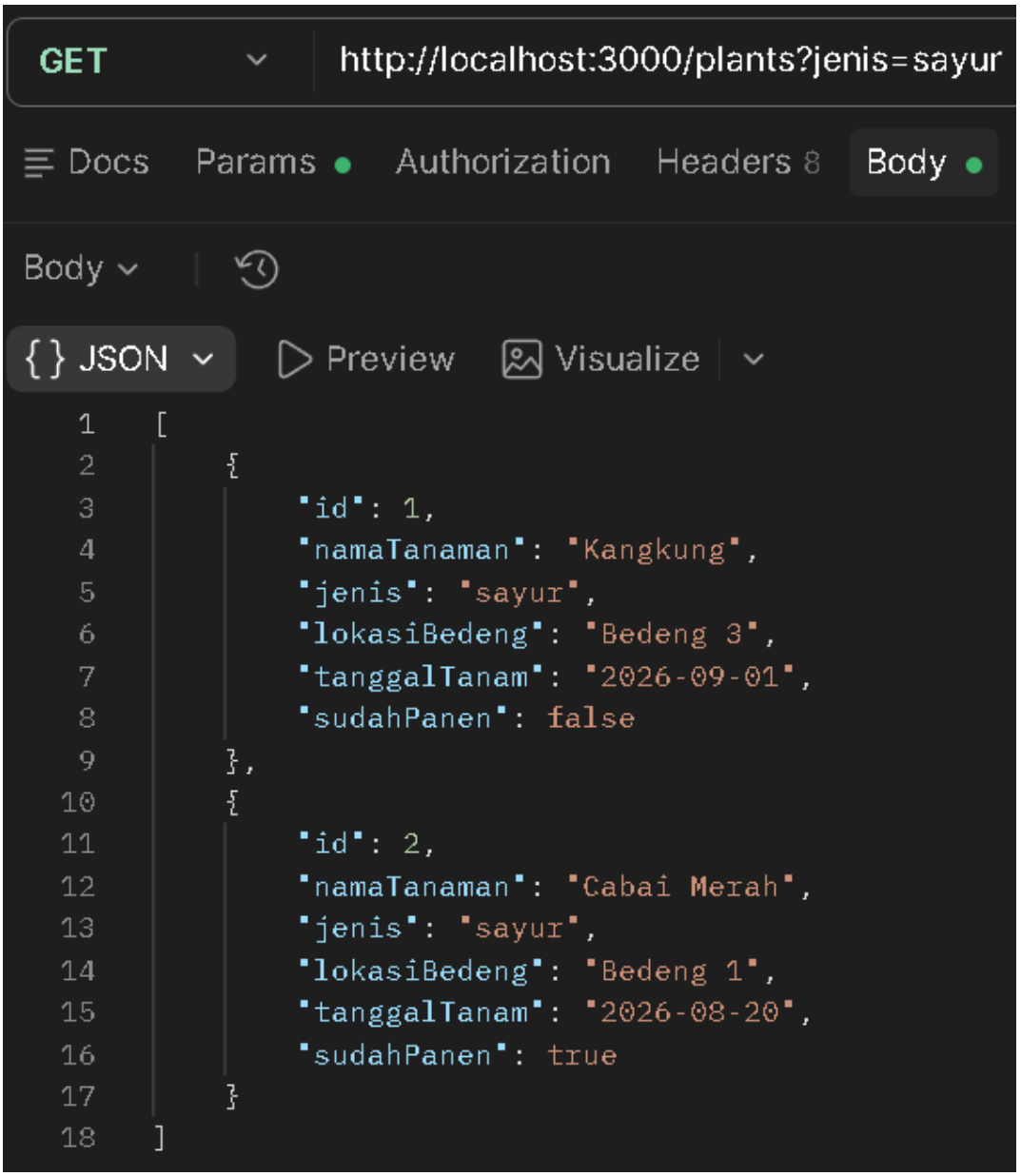
Endpoint: GET /plants?jenis=sayur

Parameter yang digunakan adalah: jenis

Contoh request: http://localhost:3000/plants?jenis=sayur

Request tersebut akan mengembalikan tanaman yang memiliki nilai jenis berupa sayur.

Screenshot 8. Filter tanaman



4.7 POST /plants

Endpoint: POST /plants

digunakan untuk menambahkan data tanaman baru.

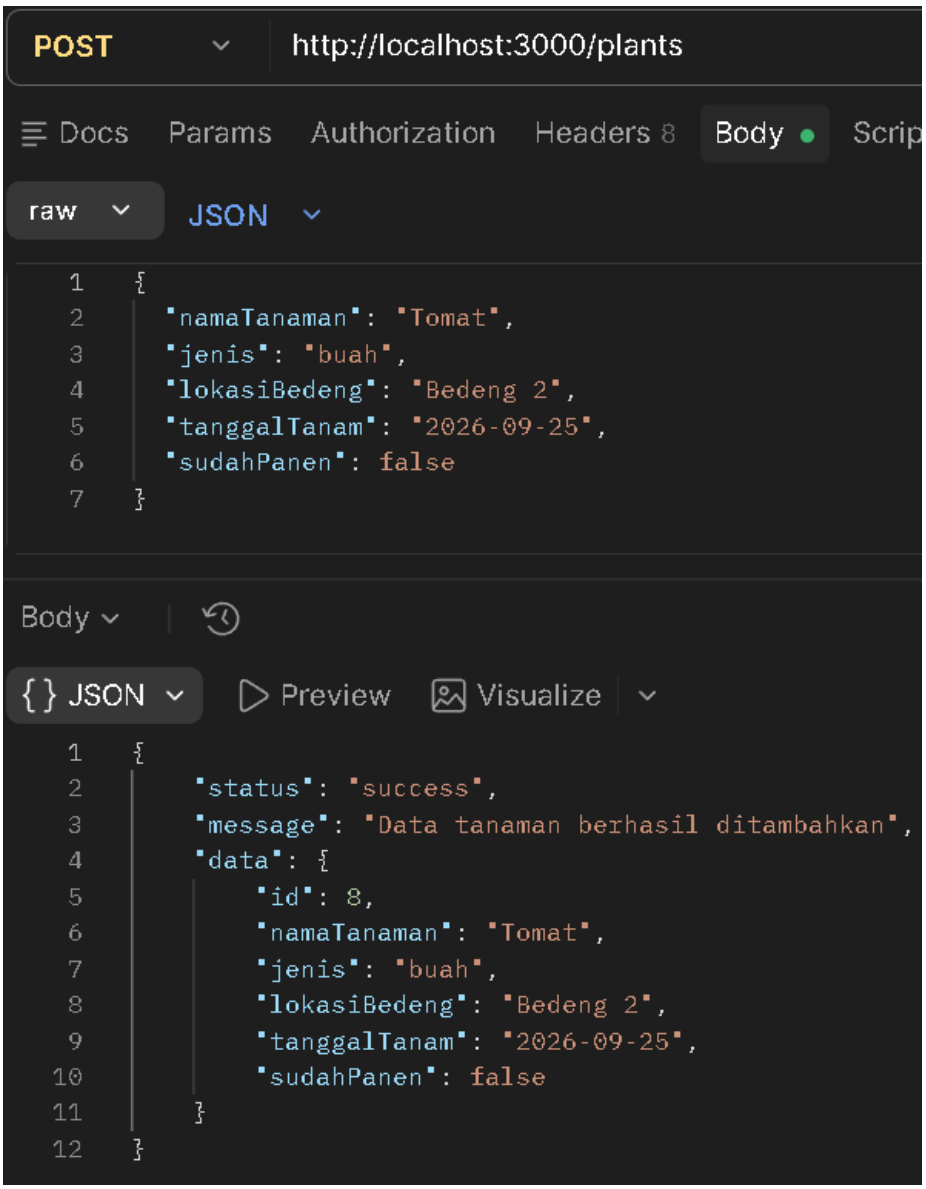
Contoh request body:

```
{
 "namaTanaman": "Tomat",
 "jenis": "buah",
 "lokasiBedeng": "Bedeng 2",
 "tanggalTanam": "2026-09-25",
 "sudahPanen": false
}
```

Data yang ditambahkan akan memperoleh ID baru secara otomatis.

Apabila data berhasil ditambahkan, API memberikan status 201 Created.

Screenshot 9. POST /plants berhasil



4.8 Validasi POST

Endpoint POST juga memiliki validasi terhadap field wajib.

Field yang harus diberikan adalah:

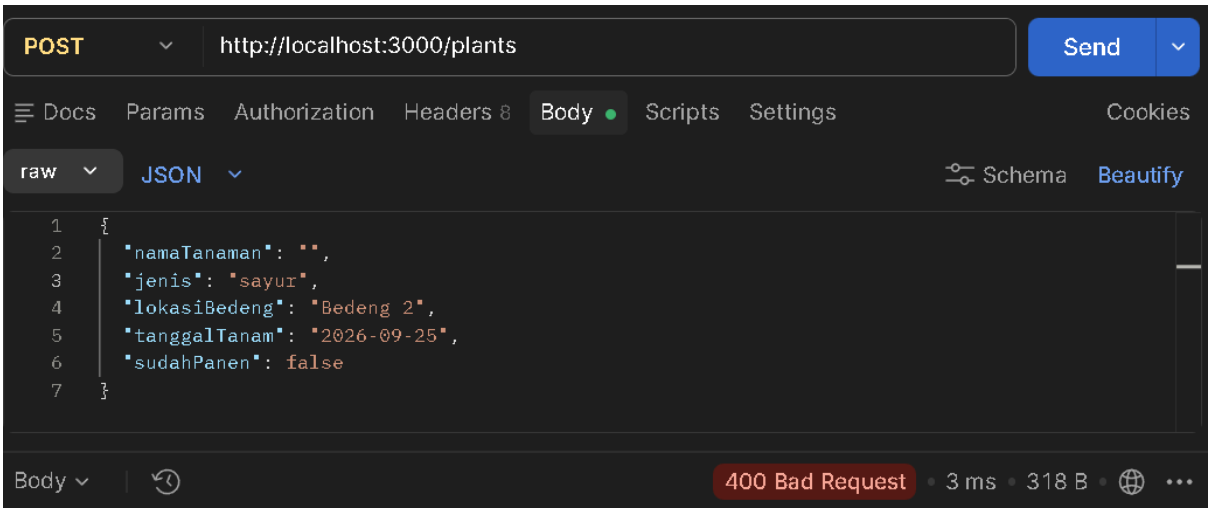
- namaTanaman
- jenis
- lokasiBedeng
- tanggalTanam

Apabila salah satu field wajib tidak diberikan atau kosong, API mengembalikan status: ***400 Bad Request***

Contoh request yang tidak valid:

```
{
 "namaTanaman": "",
 "jenis": "sayur",
 "lokasiBedeng": "Bedeng 2",
 "tanggalTanam": "2026-09-25"
}
```

Screenshot 10. POST dengan data tidak valid



4.9 PUT /plants/:id

Endpoint: PUT /plants/:id

digunakan untuk mengubah data tanaman berdasarkan ID.

Contoh: PUT /plants/1

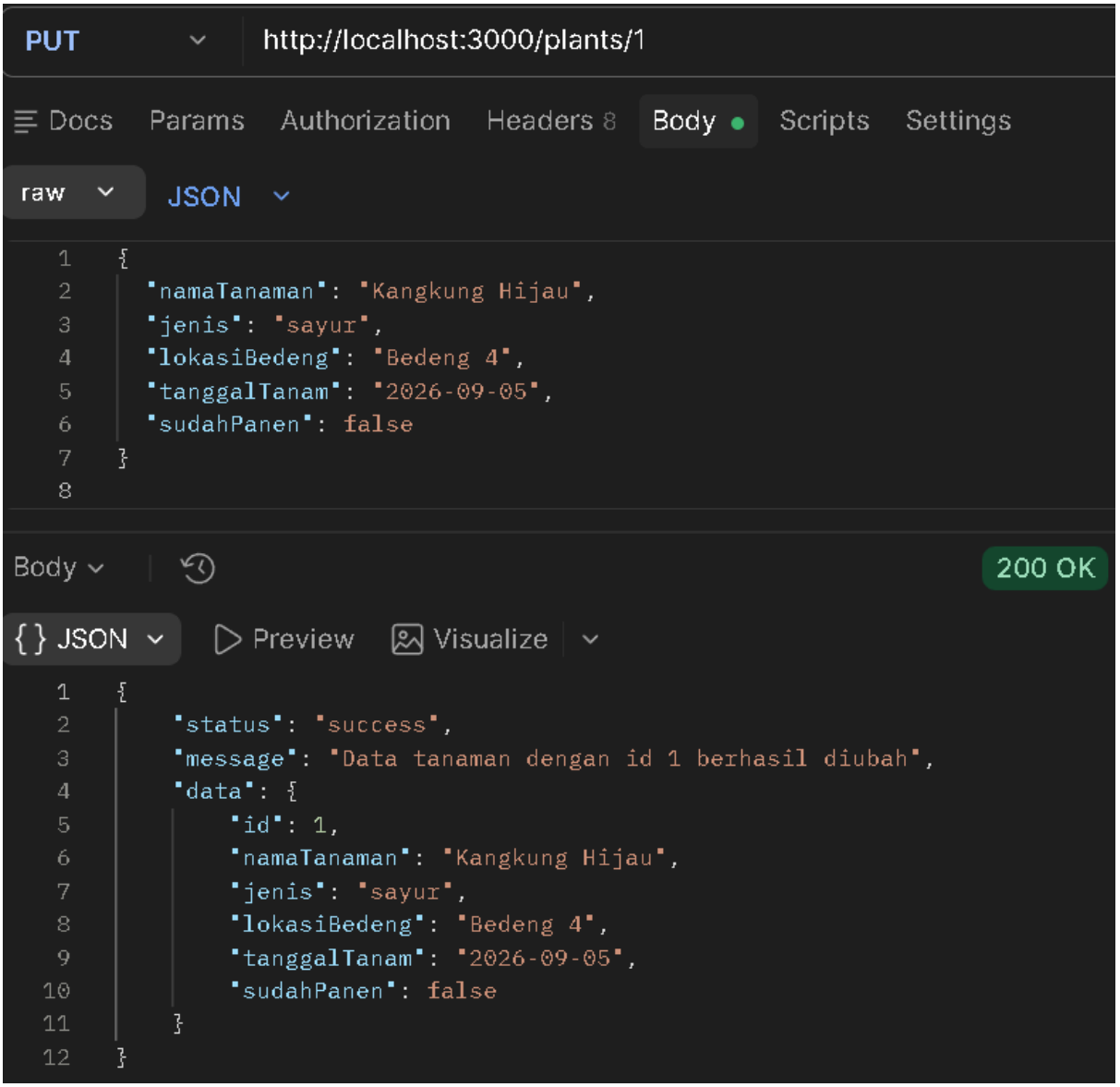
Request body:

```
{
 "namaTanaman": "Kangkung Hijau",
 "jenis": "sayur",
```

```
  "lokasiBedeng": "Bedeng 4",
  "tanggalTanam": "2026-09-05",
  "sudahPanen": false
}
```

Apabila data ditemukan dan berhasil diubah, API memberikan status 200 OK.

Screenshot 11. PUT /plants/



4.10 DELETE /plants/:id

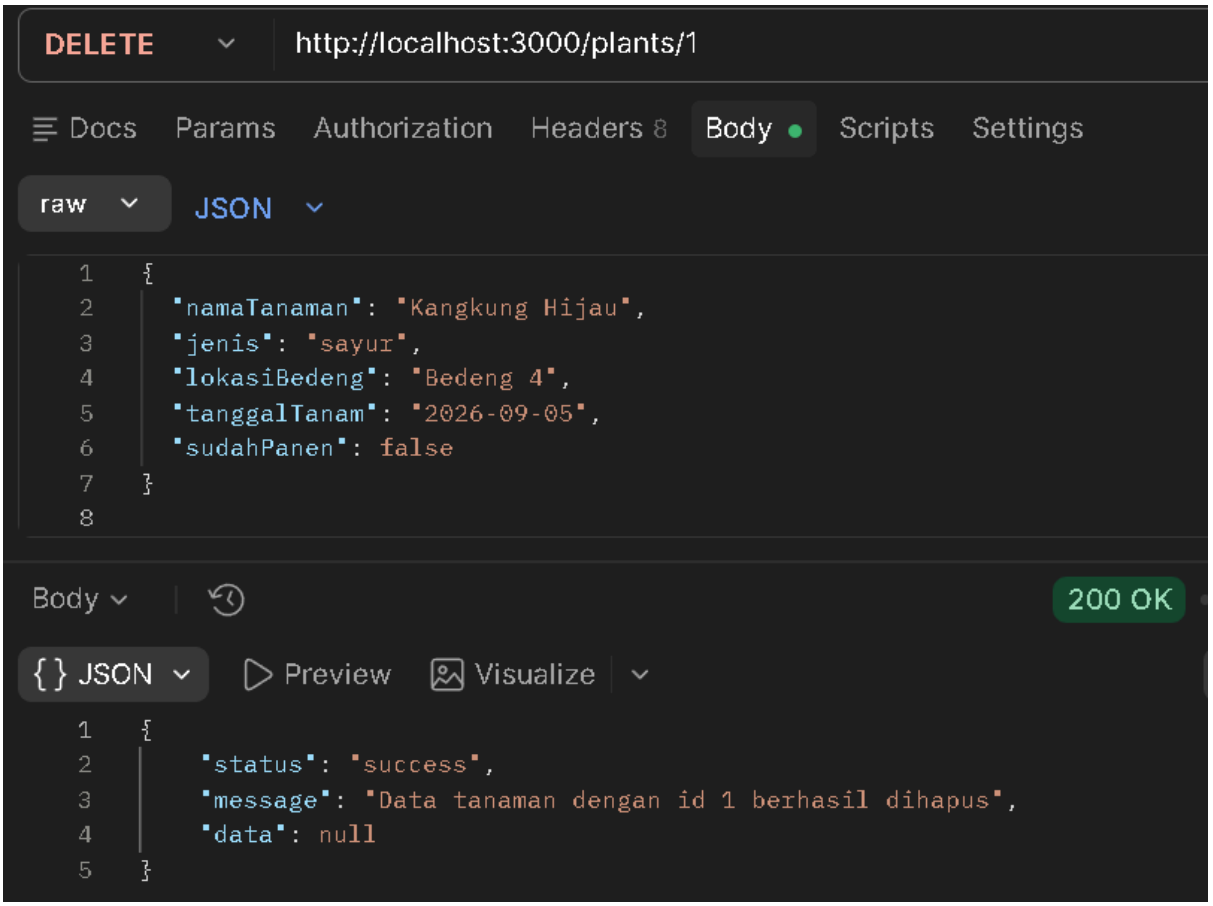
Endpoint: DELETE /plants/:id

digunakan untuk menghapus data tanaman berdasarkan ID.

Contoh: DELETE /plants/1

Apabila data ditemukan dan berhasil dihapus, API memberikan response JSON dengan status 200 OK.

Screenshot 12. DELETE /plants/



4.11 Middleware 404

API juga memiliki middleware catch-all untuk menangani endpoint yang tidak tersedia.

Contoh request: GET /alamat-tidak-ada

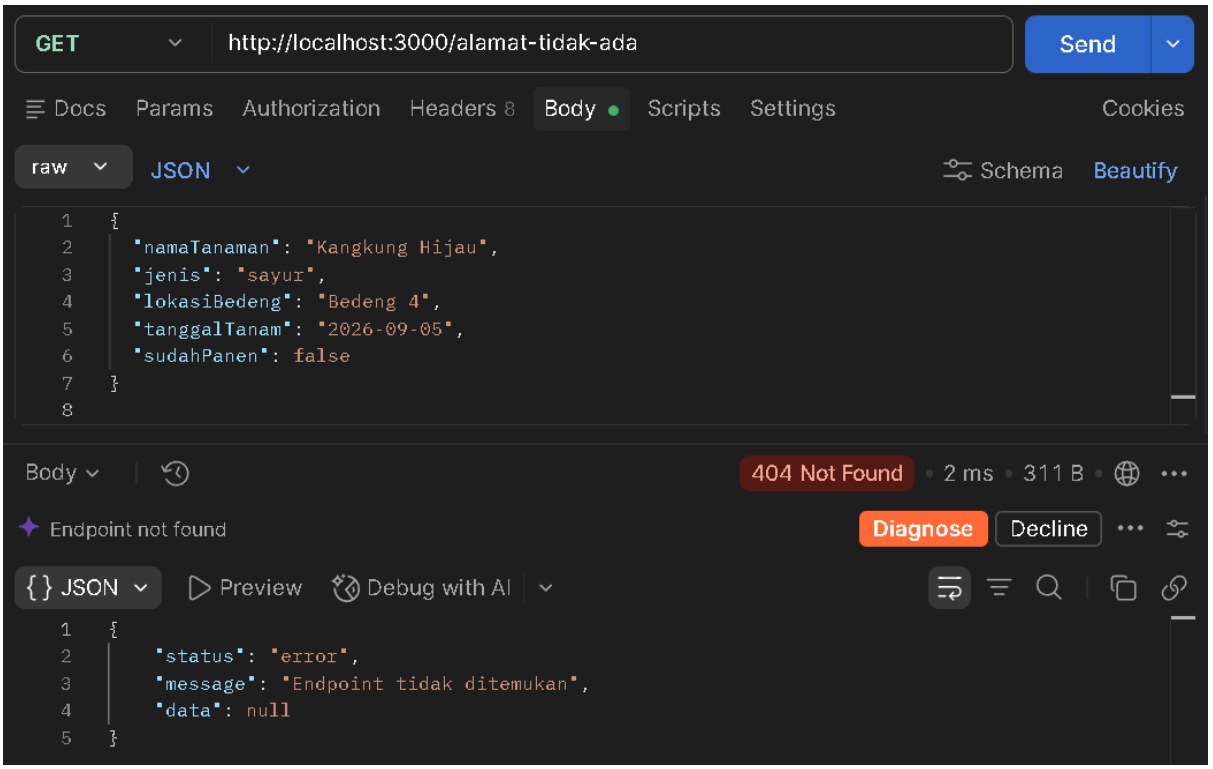
API akan memberikan status: ***404 Not Found***

dengan response dalam format JSON.

Screenshot 13. Middleware 404 pada app.js

```
290   // Catch-all 404
291   app.use((req, res) => {
292       res.status(404).json({
293           status: "error",
294           message: "Endpoint tidak ditemukan",
295           data: null
296       });
297   });
```

Screenshot 14. Pengujian endpoint yang tidak tersedia



4.12 Konfigurasi Deployment Vercel

Aplikasi dikonfigurasi agar dapat dijalankan pada Vercel.

File vercel.json digunakan untuk mengatur deployment aplikasi.

Selain itu, app.js menggunakan module.exports = app agar aplikasi dapat digunakan sebagai serverless function.

Screenshot 15. vercel.json



4.13 Push Project ke GitHub

Repository GitHub dibuat dengan nama:

tugas1-restful-2428240054

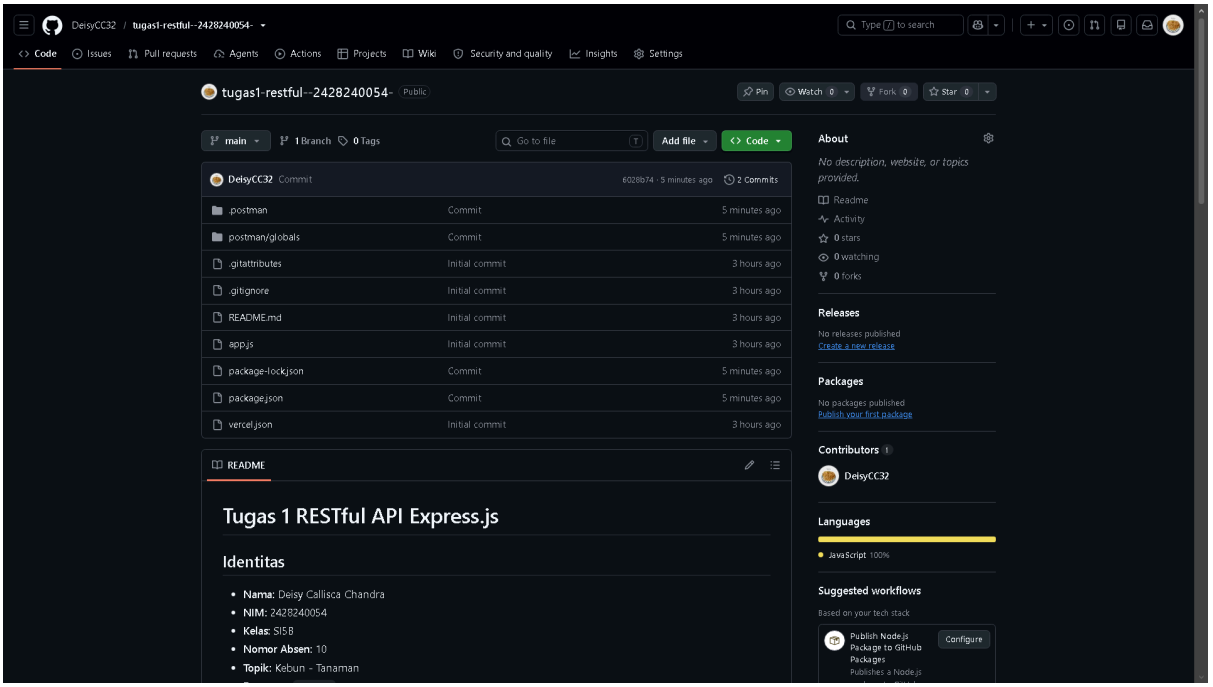
Repository dibuat dengan status **Public**.

Project kemudian di-push ke repository GitHub menggunakan Git.

Repository GitHub:

https://github.com/DeisyCC32/tugas1-restful--2428240054-

Screenshot 16. Repository GitHub



4.14 Riwayat Commit

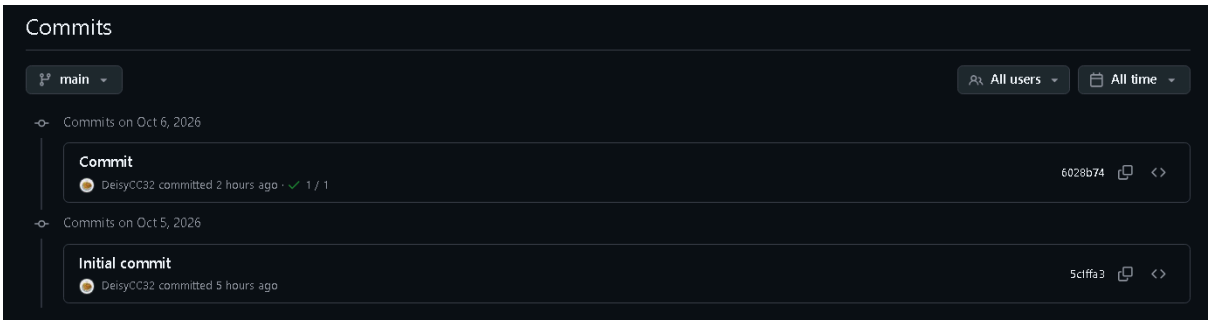
Project dibuat menggunakan commit bertahap dengan minimal lima commit.

Riwayat commit yang digunakan:

init project express
tambah route GET
tambah POST dan validasi
tambah PUT dan DELETE
konfigurasi vercel

Riwayat commit dapat diperiksa menggunakan: git log --oneline

Screenshot 17. Riwayat commit Git



4.15 Deployment ke Vercel

Repository GitHub kemudian dihubungkan dengan Vercel.

Setelah proses deployment berhasil, aplikasi dapat diakses melalui URL Vercel.

URL deployment:

https://vercel.com/figma11/tugas1-restful-2428240054

Contoh: https://tugas1-restful-2428240054.vercel.app

Screenshot 18. Deployment berhasil di Vercel

